

LOJA ESCOLA: SEU PRIMEIRO E-COMMERCE

Guia de duas aulas para abrir, entender e personalizar um projeto-modelo.

Nosso desafio do mês: criar um protótipo de loja virtual em grupo. O projeto ZIP já contém uma loja de demonstração. Primeiro vamos executá-la; depois cada grupo decide o tipo de loja e propõe suas melhorias. **Não será necessário programar tudo do zero.**

01 Como funciona uma aplicação web?

PARTE	O QUE FAZ	NESTE MOLDE
Frontend	Página que o usuário enxerga e utiliza.	HTML + CSS + JavaScript.
Backend	Recebe pedidos da página e executa as regras.	Java + Spring Boot.
Dados	Armazena informações que devem continuar existindo.	H2, um banco SQL local.

PÁGINA HTML → pede GET /api/produtos
JAVA → consulta a tabela de produtos
SQL → devolve os dados ao Java
JAVA → envia JSON à página
HTML → mostra os produtos ao usuário

02 O que o ZIP já tem?

Catálogo com três produtos, endpoint de consulta, formulário de cadastro, banco SQL local e carrinho de **demonstração**. O carrinho não faz pedidos e não realiza compras reais.

Liberdade: Java/Spring e H2 são escolhas para este molde, não uma obrigação para todos os grupos. É possível propor tecnologias equivalentes, desde que as três responsabilidades sejam explicadas.

Para conversar com o grupo

Que tipo de loja vamos criar? Papelaria, jogos, roupas, livros ou outro tema? Pense em três produtos e em uma cor principal para começar.

Nome provisório da loja: _____

Três produtos que queremos vender: _____

AULA 1 — PREPARAR E EXECUTAR

Meta: conseguir abrir a loja e identificar a página e a API.

03 Preparação (antes de abrir o código)

A primeira execução exige **internet** para baixar bibliotecas do Java. Confirme com o professor o que já está instalado no laboratório.

PRECISAMOS	COMO CONFERIR
JDK 17 ou superior	No terminal, digite java -version e javac -version . Os dois devem funcionar.
Editor de código (sugestão)	VS Code ou outro editor capaz de abrir a pasta do projeto.
Maven	O atalho Windows usa Maven instalado ou baixa uma cópia local. Se a rede bloquear, avise o professor.

04 Passo a passo para a primeira execução

Passo 1. Extraia completamente o arquivo **PROJETO_ECOMMERCE_MODELO_REVISADO.zip**. Entre na pasta **LOJA_ESCOLA_REVISADA**, onde está **pom.xml**. Não execute dentro do ZIP.

Passo 2. Abra essa pasta no editor. Encontre **src**, **pom.xml** e **INICIAR_NO_WINDOWS.bat**.

Passo 3 (Windows). Com o JDK já instalado, abra **INICIAR_NO_WINDOWS.bat**. Se o Maven já estiver configurado, também pode usar o terminal:

```
mvn spring-boot:run
```

Passo 4. Espere a mensagem **Started LojaApplication**. A primeira execução poderá levar alguns minutos. Deixe a janela do servidor aberta.

Passo 5. No navegador do mesmo computador, abra:

```
http://localhost:8080
```

Deu certo? Aparece a página Loja Escola com Camiseta, Caneca e Caderno. Se só aparecer erro ou a página não carregar, volte ao terminal e peça ajuda antes de alterar o código.

Checklist da instalação

- ☐ O ZIP foi extraído. ☐ O JDK foi reconhecido. ☐ O Java iniciou sem erro.
- ☐ A página abriu no navegador. ☐ Eu encontrei o arquivo pom.xml.

Se aparecer um erro

Copie a **última mensagem de erro do terminal** antes de modificar arquivos. Isso ajuda a descobrir se faltou o JDK, a internet ou a porta 8080 está ocupada.

AULA 1 — OLHAR E PERSONALIZAR

Meta: fazer duas mudanças simples sem editar o Java.

05 Seu primeiro teste da API

Abra uma **segunda aba** do navegador e acesse:

`http://localhost:8080/api/produtos`

Você verá os mesmos produtos, mas em **JSON** (texto estruturado). A página usa JavaScript para receber esses dados e criar os cartões do catálogo.

```
[{"id":1,"nome":"Camiseta da escola",
  "preco":39.90,"estoque":12}, ...]
```

Observação: os números de ID e a ordem dos produtos podem variar; isso é normal. O importante é encontrar nome, preço e estoque.

06 Duas alterações simples do grupo

ALTERAÇÃO	ONDE MEXER	COMO TESTAR
Nome da loja	Abra src/main/resources/static/index.html e troque o texto do título Loja Escola entre as tags <code>h1</code> .	Salve e recarregue a página.
Cor principal	Abra src/main/resources/static/estilo.css e troque o valor de --cor-principal .	Salve e atualize a página.

```
/* Troque APENAS este valor dentro da regra :root */
--cor-principal: #b5121b; /* antes */
--cor-principal: #1759a8; /* depois */
```

Usando o iniciador Windows ou **mvn spring-boot:run**, alterações em HTML/CSS podem aparecer ao recarregar. Se houver cache, pressione **Ctrl+F5**. Se iniciou diretamente pelo editor e não mudou, reinicie o Java.

07 Evidências da primeira aula

Guarde **2 prints**: (1) a página da loja com as modificações do grupo; (2) o JSON em **/api/produtos**. Não é necessário entregar código Java nesta etapa.

Se a escola bloquear a instalação: abra **PLANO_B_SEM_JAVA.html** com um navegador e personalize o HTML. Essa é apenas uma maquete sem API ou SQL. O professor retomará a integração em outro encontro.

Para discutir em dupla

Por que o JSON tem os mesmos produtos da página, mas não possui cores, imagens ou botões? Onde entram o JavaScript e o CSS nessa diferença?

AULA 2 — HTML, JAVA E SQL

Meta: ver um produto sair do formulário, passar pela API e chegar ao banco.

08 O que acontece ao cadastrar?

FORMULÁRIO HTML → JavaScript faz POST
 JAVA → valida e salva um Produto
 H2 → armazena a nova linha
 JAVA → confirma o cadastro
 HTML → atualiza o catálogo

O endereço é o mesmo **/api/produtos**. O que muda é o método HTTP: **GET** consulta os produtos; **POST** envia um novo produto.

09 Exercício guiado: cadastrar

Passo 1. Com a loja funcionando, role até **Laboratório: cadastrar produto**.

Passo 2. Preencha um exemplo: nome = Estojo, preço = 19.90 e estoque = 5. Clique em **Cadastrar produto** uma vez.

Passo 3. Veja o novo produto aparecer na página. Atualize a outra aba em **/api/produtos** e localize o nome Estojo no JSON.

Passo 4. Pare o servidor com **Ctrl+C** e inicie novamente. Se você usou a mesma pasta do projeto, o produto deverá continuar lá: o banco H2 foi salvo em disco.

10 O carrinho também fica salvo no banco?

Não. Adicione um produto ao carrinho e atualize a página. O carrinho ficará vazio outra vez, pois é só uma variável temporária do JavaScript. Isso mostra a diferença entre **dados persistidos** e **dados temporários**.

Atenção: o cadastro está aberto para testes e não possui login. Esta aplicação foi feita para executar SOMENTE no computador da escola, sem informações pessoais ou pagamentos reais.

11 Opcional: consulte o SQL diretamente

Com o Java ligado, acesse **http://localhost:8080/h2-console**. Use URL de conexão **jdbc:h2:file:./dados/loja**, usuário **sa** e senha vazia. Clique em Connect e execute:

```
SELECT * FROM PRODUTO;
```

Encontre o Estojo na tabela. Esse passo é opcional para a aula 2 e poderá ser retomado depois.

Registro da aula 2

Qual era o produto cadastrado? _____

Depois de reiniciar, ele continuou aparecendo? _____

O carrinho também permaneceu cheio? Qual é a diferença entre esses dois dados?

O PROJETO E OS CONCEITOS DE POO

Você não precisa decorar os arquivos: aprenda a função de cada um.

12 Mapa dos arquivos principais

ARQUIVO	FUNÇÃO DIDÁTICA
index.html	Estrutura da página: textos, seções e formulário.
estilo.css	Aparência: cores, espaçamentos e cartões.
loja.js	Faz GET e POST: liga a página à API Java.
Produto.java	Classe que define os atributos de cada produto.
ProdutoController.java	Métodos que atendem às requisições HTTP.
ProdutoRepository.java	Interface fornecida pelo Spring para acessar o SQL.
application.properties	Endereço do banco e configurações do servidor.

13 Identifique os conceitos já estudados

Classe: Produto é o molde. **Objeto:** cada camiseta ou caderno cadastrado é um Produto criado a partir desse molde.

Encapsulamento: os atributos da classe são **private**; métodos como **getNome()** permitem consultá-los.

Interface: **ProdutoRepository** é declarada como interface e estende uma interface pronta do Spring. Isso permite usar operações como listar e salvar sem implementar tudo agora.

Método: **listar()** e **cadastrar()** ficam no Controller e atendem as solicitações da página.

Nesta etapa, classes abstratas, herança e polimorfismo podem ser estudados separadamente; **não é necessário forçar esses conceitos neste protótipo inicial.**

Trecho de código para identificar

```
private String nome;           // atributo encapsulado
public String getNome() {      // método de leitura
    return nome;
}
```

Localize esse trecho em **Produto.java**. Que informações estão escondidas como **private**? Qual é o objetivo de permitir a consulta pelo getter?

ACOMPANHAMENTO DO PROJETO

Um caminho sugerido para o mês, adaptável à realidade de cada grupo.

14 O que desenvolver depois?

ETAPA	POSSIBILIDADE
Primeiros dias	Fazer o molde funcionar, escolher um tipo de loja, mudar nome e visual.
Integração	Entender GET/POST; cadastrar um produto e comprovar persistência.
Melhoria simples	Escolher uma evolução: busca por nome, categoria ou outra ideia viável.
Planejamento final	Desenhar clientes e pedidos no papel; implementar só se o grupo conseguir.

15 Registro sugerido

O grupo pode reunir em **um PDF**: print da loja personalizada; print do JSON; print de um produto cadastrado (na segunda aula); e respostas curtas: **qual é o frontend? qual é o backend? onde os dados são armazenados? por que o carrinho se perde quando recarregamos a página?** O professor pode adaptar a entrega.

16 Se alguma coisa der errado...

SITUAÇÃO	O QUE CONFERIR
O Java não inicia	Teste java -version e javac -version. Precisa de JDK 17+.
Erro ao baixar Maven	Pode ser internet, proxy ou política da escola. Use Plano B e informe o professor.
localhost não abre	Deixe o terminal aberto; aguarde Started LojaApplication. Veja a porta.
A página não muda	Salve; Ctrl+F5. Se rodou direto pelo editor, tente reiniciar o Java.
Produtos de exemplo não mudam	Cargal inicial só popula um banco vazio; cadastre pelo formulário.
O produto desapareceu	Veja se abriu o mesmo projeto/pasta. Cada cópia usa o próprio banco H2.

Regra do projeto: façam uma mudança por vez, testem e registrem o resultado. Este molde não é uma loja pronta para uso comercial: não possui controle de acesso, compras ou pagamentos reais.

Próxima conversa do grupo

Uma melhoria que queremos tentar: _____

Como saberemos que funcionou? _____